

Ergebnisbericht — Smart-Home-Demonstrator

Heterogeneous Computing, Übungsblatt 1 (Aufgabe 1b), Sommersemester 2026

1. Ziel und Vorgehen

Ziel der Arbeit ist der Entwurf und die prototypische Umsetzung einer modernen Smart-Home-Infrastruktur, die zentrale Eigenschaften heterogener verteilter Systeme (Verteilung, Skalierbarkeit, Fehlertoleranz, Robustheit, lose Kopplung, Heterogenität und Erweiterbarkeit) nicht nur abstrakt veranschaulicht, sondern konkret demonstriert.

Methodisch baut die Arbeit aufeinander auf: Zunächst werden bestehende Architekturen, Plattformen und Protokolle recherchiert (Abschnitt 2). Aus den gewonnenen Erkenntnissen werden Anforderungen abgeleitet (Abschnitt 3) und zu einer Architektur verdichtet (Abschnitt 4). Diese wird als lauffähiger Demonstrator implementiert (Abschnitt 5), wobei jede wesentliche Entwurfsentscheidung auf einen Recherchebefund zurückgeführt wird. Abschnitt 6 bewertet das Ergebnis und stützt sich dabei auf einen automatisierten Akzeptanztest. Physische Hardware wird nicht eingesetzt. Sensoren und Aktoren werden simuliert durch realistisch verhaltende Software-Mockups.

2. Recherche: Smart-Home-Architekturen und -Technologien

Heterogenität ist im Smart-Home-Umfeld der Normalfall: Geräte unterschiedlicher Hersteller, Rechenleistung und Funktechnik müssen zusammenwirken, weshalb Interoperabilität als eine der zentralen Herausforderungen des IoT gilt [2]. Die folgende Recherche ordnet das Feld entlang der vier Teilen Kommunikationsmodelle, Protokolle, Plattformen sowie aktuellen Trends.

2.1 Kommunikationsmodelle

Drei Modelle prägen die Kommunikation im Smart Home. Das Anfrage-Antwort-Modell (Request/Response) eignet sich für gezielte Abfragen und Steuerbefehle, koppelt Sender und Empfänger jedoch zeitlich und räumlich aneinander: Beide müssen gleichzeitig verfügbar sein und einander adressieren. Das Publish/Subscribe-Modell hebt diese Kopplung auf und entkoppelt Erzeuger und Verbraucher in drei Dimensionen. Im Raum (die Teilnehmer kennen einander nicht), in der Zeit (sie müssen nicht gleichzeitig aktiv sein) und in der Synchronisation (das Senden blockiert den Sender nicht). Gerade diese dreifache Entkopplung macht das Modell für lose gekoppelte, großskalige verteilte Systeme geeignet [1]. Das ereignisgetriebene Modell baut darauf auf, indem Komponenten ausschließlich auf eintreffende Ereignisse reagieren. Für IoT-Szenarien mit vielen, häufig wechselnden Teilnehmern hat sich Publish/Subscribe durchgesetzt, weil es n:m-Kommunikation erlaubt und Teilnehmer ohne Rekonfiguration des Gesamtsystems hinzukommen oder wegfallen können.

2.2 Protokolle und Standards

Auf der Anwendungsschicht ist MQTT das verbreitetste Protokoll: ein leichtgewichtiges Publish/Subscribe-Protokoll über TCP mit gestaffelten Quality-of-Service-Stufen, dauerhaft gespeicherten Nachrichten (retained messages) und einem Testament-Mechanismus (Last Will and Testament) zur Ausfallerkennung [6]. CoAP ist ein an HTTP angelehntes Request/Response-Protokoll über UDP und wurde gezielt für ressourcenbeschränkte Geräte und Netze entworfen [7]. Ein

systematischer Vergleich zeigt, dass kein einzelnes Protokoll alle Anforderungen gleichermaßen erfüllt. MQTT, CoAP, AMQP und HTTP unterscheiden sich charakteristisch in Nachrichtengröße, QoS-Garantien und Interoperabilität [3]. Messungen über eine gemeinsame Middleware bestätigen das Bild: MQTT spielt seine Stärken bei zuverlässiger 1:n-Verteilung aus, während CoAP bei sehr knapper Bandbreite und auf besonders ressourcenarmen Knoten geringeren Overhead verursacht [4].

Auf der Vernetzungsschicht koexistieren die etablierten Mesh-Funkstandards Zigbee und Z-Wave mit Thread, einem IPv6-fähigen Low-Power-Mesh, in dem jedes Gerät als Repeater wirken kann. Eine Sonderrolle nimmt Matter ein: Es konkurriert nicht auf der Funkebene, sondern bildet eine herstellerübergreifende Anwendungs- und Datenmodellschicht auf Basis von IPv6 und modelliert Geräte als Sammlung von Clustern mit Attributen und Kommandos [8].

2.3 Plattformen, Cloud und Edge

Auf Plattformseite stehen offene, lokal betriebene Orchestratoren wie Home Assistant und openHAB den cloud-zentrierten Ökosystemen großer Anbieter gegenüber. Der entscheidende Architekturgegensatz ist cloud-zentriert gegenüber local-first. Die Verlagerung der Verarbeitung näher an die Datenquelle (Edge Computing) senkt Latenz und Bandbreitenbedarf und verbessert den Schutz der Privatsphäre, da Rohdaten das lokale Netz nicht verlassen müssen [5]. Ein lokal arbeitender Hub bleibt zudem bei Ausfall der Internetverbindung funktionsfähig.

2.4 Aktuelle Trends (2026)

Das Feld bewegt sich entlang dreier Linien. Erstens setzt sich die Konvergenz auf offene Standards fort: Mit Matter und Thread verschiebt sich der Wettbewerb von Ökosystem-Bindung hin zu Gerätequalität und Interoperabilität [8]. Zweitens verlagert sich Intelligenz an den Rand (Edge-AI): Maschinelles Lernen läuft zunehmend direkt auf dem Hub, etwa für vorausschauende Automatisierung, als konsequente Fortführung des Edge-Gedankens [5]. Drittens gewinnt Energiemanagement an Bedeutung. Smarte Thermostate und Energiemonitoring zählen zu den am stärksten wachsenden Gerätekategorien.

3. Abgeleitete Anforderungen

Aus der Recherche ergeben sich sieben Anforderungen, die zugleich die Bewertungsgrundlage bilden:

- **Verteilung** — Komponenten laufen als eigenständige Prozesse und kommunizieren ausschließlich über das Netz.
- **Skalierbarkeit** — neue Geräte und Instanzen lassen sich ohne Umbau des Kerns ergänzen; dies folgt aus dem n:m-Charakter von Publish/Subscribe [1].
- **Fehlertoleranz** — der Ausfall eines Geräts oder der Cloud legt das System nicht lahm; gestützt durch lokalen Edge-Betrieb [5] und die MQTT-Ausfallerkennung [6].
- **Robustheit** — definierte Schnittstellen, Validierung und Wiederverbindung fangen Fehlersituationen ab.
- **Lose Kopplung** — Sender und Empfänger kennen einander nicht direkt [1].
- **Heterogenität** — Geräte unterschiedlicher Klassen und Fähigkeiten arbeiten über einheitliche Schnittstellen zusammen [2].

- **Erweiterbarkeit** — neue Gerätetypen docken über den Schnittstellen-Vertrag an.

4. Architektur des Demonstrators

Leitbild ist ein ereignisgetriebener Message-Bus als Nervensystem: Ein zentraler MQTT-Broker entkoppelt alle Teilnehmer. Diese Wahl folgt unmittelbar aus der dreifachen Entkopplung des Publish/Subscribe-Modells [1] und der Eignung von MQTT für zuverlässige 1:n-Verteilung [4]. Geräte veröffentlichen Telemetrie und Zustände, abonnieren Kommandos und kommunizieren niemals direkt miteinander. Die fachliche Logik, also Automatisierung und Geräteverwaltung (Discovery) sitzt am Edge und arbeitet auch ohne Cloud weiter [5]. Eine Cloud-Komponente ist als optionale Erweiterung vorgesehen.

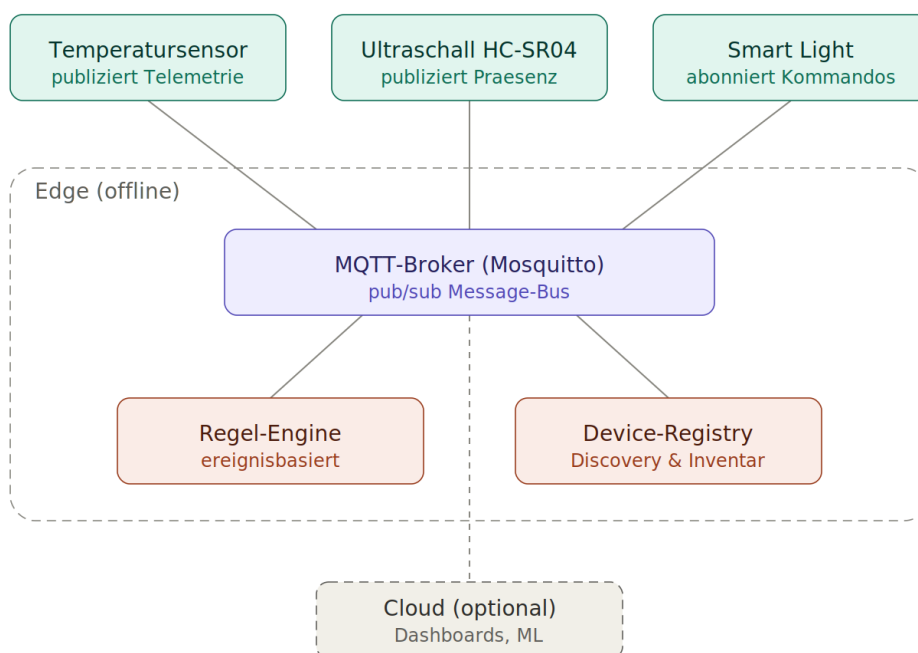


Abb. 1: Architektur des Demonstrators (Message-Bus als Nervensystem).

Nachrichtenfluss am Beispiel: Der Ultraschallsensor publiziert eine Distanzmessung auf sein Telemetrie-Topic. Die Regel-Engine ist darauf abonniert, leitet daraus Anwesenheit ab und publiziert bei einem Zustandswechsel ein Kommando auf das Kommando-Topic der zugehörigen Lampe. Die Lampe schaltet und meldet ihren neuen Zustand zurück. Parallel führt die Registry aus dauerhaft gespeicherten Anmelde- und Verfügbarkeitsnachrichten ein Live-Inventar.

5. Implementierung

Dieser Abschnitt überführt die Architektur in eine konkrete Umsetzung auf Basis der Anforderungen.

5.1 Technologie und Aufbau

Der Demonstrator ist in Python umgesetzt (MQTT-Anbindung über paho-mqtt), nutzt Eclipse Mosquitto als Broker und wird über Docker Compose orchestriert. Die Wahl von MQTT als Anwendungsprotokoll ergibt sich direkt aus der Recherche: Für die hier benötigte 1:n-Verteilung von

Ereignissen an mehrere Verbraucher ist es das geeignete Protokoll [3][4]. Dass jede Komponente (Broker, Sensoren, Aktor, Regel-Engine, Registry und Dashboard) als eigener Container läuft, setzt die Anforderung Verteilung unmittelbar um: Es gibt keine gemeinsamen Speicherbereiche, jede Komponente ist ein eigenständiger Prozess, der ausschließlich über das Netz kommuniziert.

5.2 Schnittstellen-Vertrag

Die Schnittstelle ist explizit und maschinenprüfbar definiert: ein hierarchisches Topic-Schema `home/{area}/{device_type}/{device_id}/{channel}` mit den Kanälen `telemetry`, `state`, `command` und `availability` sowie ein Discovery-Topic `home/_registry/announce/{id}`. Für jeden Nachrichtentyp existiert ein JSON-Schema, gegen das jede Nachricht vor dem Versand validiert wird. Dieser Vertrag ist der einzige geteilte Wissensstand zwischen den ansonsten unabhängigen Komponenten und damit die technische Grundlage für Interoperabilität zwischen heterogenen Geräten [2]. Die attribut- und kommandobasierte Struktur orientiert sich bewusst am Cluster-Datenmodell von Matter [8], ohne dessen vollen Funktionsumfang nachzubilden.

5.3 Lose Kopplung, Automatisierung und Discovery

Die fachliche Logik ist auf zwei Edge-Dienste verteilt, die die strukturell vorgegebene lose Kopplung [1] in konkretes Verhalten überführen. Die Regel-Engine abonniert Telemetrie, leitet Anwesenheit ab und steuert Aktoren ausschließlich über Kommando-Topics. Sie kennt die gesteuerten Geräte nicht statisch, sondern lernt sie zur Laufzeit aus den Discovery-Nachrichten der Registry. Dadurch lässt sich ein neues Gerät allein durch sein Erscheinen ohne Codeänderung am Bus integrieren. Die Anforderungen lose Kopplung und Erweiterbarkeit werden so unmittelbar wirksam. Die Registry führt aus dauerhaft gespeicherten Anmelde- und Verfügbarkeitsnachrichten ein Live-Inventar. Der Testament-Mechanismus von MQTT [6] meldet den Ausfall eines Geräts automatisch und trägt so zur Fehlertoleranz bei.

5.4 Robustheit

Robustheit ist nicht nur als Schnittstellendefinition vorhanden, sondern aktiv abgesichert: Eingehende Nachrichten werden gegen ihr JSON-Schema geprüft, und nicht dekodierbare, syntaktisch fehlerhafte oder unerwartet geformte Nachrichten werden verworfen, ohne den verarbeitenden Dienst zu beeinträchtigen. Verbindungsabbrüche werden durch eine Wiederverbindungslogik aufgefangen. Diese defensive Verarbeitung stellt sicher, dass einzelne fehlerhafte Teilnehmer die Regelschleife nicht unterbrechen.

5.5 Simulierte Geräte (Mockups) und Einsatz generativer KI

Anstelle physischer Hardware kommen Software-Mockups zum Einsatz: ein Temperatursensor, ein Ultraschall-Abstandssensor (HC-SR04) sowie eine schaltbare Lampe als Aktor. Damit sich die Sensoren möglichst realistisch verhalten, erzeugen sie ihre Werte nicht zufällig, sondern aus vorab durch generative KI (Claude Opus 4.8) erzeugten Tagesprofilen. Die Ergebnisse wurden geprüft und als JSON abgelegt. Je Raum und Tageszeit enthalten sie Anwesenheit und Temperatur, die die Sensoren zeitabhängig abspielen. Die generative KI läuft bewusst vorab und nicht im Live-Betrieb. Die Datensätze sind versioniert, die Demonstration spielt sie nur ab. Das hält den Betrieb deterministisch und ausfallfrei und trennt die Datenerzeugung sauber von der Infrastruktur. Drei Profile liegen bei: `day_profile` (Werktag), `urlaub` (dauerhafte Abwesenheit) und `besuch` (viele Personen).

Das aktive Profil sowie eine simulierte Uhr lassen sich zur Laufzeit über dauerhaft gespeicherte Control-Topics umschalten, bedienbar per Dashboard-Button oder Kommandozeile. Alle Sensoren sind darauf abonniert und wechseln sofort. Dass selbst diese Steuerung nur aus Bus-Nachrichten besteht, ist ein weiterer Beleg für die lose Kopplung. Über die simulierte Uhr lässt sich auf eine feste Tageszeit springen oder ein kompletter Tag im Zeitraffer durchlaufen.

5.6 Heterogenität und Mehrraumbetrieb

Jedes Gerät meldet bei der Registrierung eine Gerätekategorie (constrained, edge oder cloud). Diese macht die Heterogenität explizit: ein ressourcenbeschränkter Sensorknoten (vgl. Cortex-M, in der Praxis typischerweise CoAP-fähig [4][7]) steht einem leistungsfähigeren Edge-Knoten gegenüber (vgl. Cortex-A), auf dem die Automatisierung läuft [5]. Der Demonstrator betreibt zwei Räume (living_room, bedroom) aus identischem Code. Die Instanzen unterscheiden sich allein durch eine Umgebungsvariable. Registry und Regel-Engine bleiben dabei unverändert, da sie raumweise über Topic-Wildcards arbeiten. Die Anforderungen Skalierbarkeit und Erweiterbarkeit werden so ohne Eingriff in den Kern erfüllt.

5.7 Visualisierung

Ein Web-Dashboard verbindet sich per MQTT-over-WebSockets direkt mit dem Broker und zeigt den Live-Zustand aller Geräte. Es greift auf keine interne Schnittstelle zu, sondern hängt als gewöhnlicher Subscriber am Bus. Es ließe sich entfernen oder vervielfachen, ohne dass eine andere Komponente davon berührt wäre (lose Kopplung).

6. Bewertung

Die folgende Tabelle ordnet jeder Anforderung ihre konkrete Umsetzung zu:

Anforderung	Umsetzung im Demonstrator
Verteilung	Jede Komponente ist ein eigener Prozess/Container
Skalierbarkeit	Zweiter Raum aus identischem Code, ohne Änderung am Kern
Fehlertoleranz	Last-Will markiert Ausfälle; Edge arbeitet ohne Cloud weiter
Robustheit	JSON-Schema-Validierung, Reconnect-Logik, Verwerfen ungültiger Daten
Lose Kopplung	Pub/Sub über den Broker; Sender kennen Empfänger nicht
Heterogenität	Gerätekategorie und unterschiedliche Fähigkeiten je Gerät
Erweiterbarkeit	Registry-Discovery; Regel-Engine lernt Aktoren zur Laufzeit

Die lose Kopplung ist die Grundlage, auf der die übrigen Eigenschaften aufbauen: Weil Geräte nur den Schnittstellen-Vertrag teilen, lassen sie sich unabhängig hinzufügen (Skalierbarkeit, Erweiterbarkeit) oder entfernen (Fehlertoleranz). Diese Eigenschaften werden nicht nur behauptet, sondern in einem automatisierten Akzeptanztest aktiv geprüft: Ein Szenario-Client schleust zur Laufzeit einen komplett neuen Raum allein über den Bus ein und weist nach, dass das unveränderte System ihn entdeckt und steuert (Discovery, lose Kopplung, Erweiterbarkeit), dass fehlerhafte Nachrichten die Regelschleife nicht unterbrechen (Robustheit), dass ein ausgefallener Akteur die

Engine nicht blockiert (Fehlertoleranz) und dass das System mit vielen zusätzlichen Geräten weiterarbeitet (Skalierbarkeit).

Grenzen und Ausblick. Der Demonstrator simuliert Geräte und verwendet ein vereinfachtes Verhaltensmodell; reale Funkprotokolle (Thread, Zigbee) und das vollständige Matter-Datenmodell sind nicht implementiert. Naheliegende Erweiterungen sind eine Annäherung des Datenmodells an Matter-Cluster [8], eine optionale Cloud-Komponente für persistente Auswertung sowie eine lokale Edge-AI-Komponente für vorausschauende Automatisierung [5].

Einsatz von KI-Werkzeugen bei der Bearbeitung

Bei der Bearbeitung dieser Aufgabe wurden generative KI-Werkzeuge (ein großes Sprachmodell) unterstützend eingesetzt. Die KI half beim Strukturieren der Recherche, beim Entwurf der Architektur, bei der Erzeugung und Überarbeitung des Programmcodes, bei der Erstellung des Architekturdiagramms sowie beim Verfassen und Formatieren dieses Berichts. Zusätzlich wurden die simulierten Sensordaten (die Tagesprofile in `app/data/`) vorab mit einem Sprachmodell erzeugt; dieser Einsatz ist Teil der Aufgabenstellung und in Abschnitt 5.5 beschrieben.

Die KI wurde iterativ im Dialog genutzt: Vorschläge wurden geprüft, angepasst und gegengeprüft. Der Programmcode wurde ausgeführt und getestet; die angegebenen wissenschaftlichen Quellen wurden auf tatsächliche Existenz und korrekte Angaben überprüft.

Quellen

Wissenschaftliche Quellen

- [1] Eugster, P. T.; Felber, P. A.; Guerraoui, R.; Kermarrec, A.-M. (2003): The Many Faces of Publish/Subscribe. *ACM Computing Surveys* 35(2), S. 114–131. DOI: 10.1145/857076.857078
- [2] Al-Fuqaha, A.; Guizani, M.; Mohammadi, M.; Aledhari, M.; Ayyash, M. (2015): Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials* 17(4), S. 2347–2376. DOI: 10.1109/COMST.2015.2444095
- [3] Naik, N. (2017): Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP. In: 2017 IEEE International Systems Engineering Symposium (ISSE). IEEE.
- [4] Thangavel, D.; Ma, X.; Valera, A.; Tan, H.-X.; Tan, C. K.-Y. (2014): Performance Evaluation of MQTT and CoAP via a Common Middleware. In: 2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP). IEEE. DOI: 10.1109/ISSNIP.2014.6827678
- [5] Shi, W.; Cao, J.; Zhang, Q.; Li, Y.; Xu, L. (2016): Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal* 3(5), S. 637–646. DOI: 10.1109/JIOT.2016.2579198
- [6] OASIS (2019): MQTT Version 5.0. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [7] Shelby, Z.; Hartke, K.; Bormann, C. (2014): The Constrained Application Protocol (CoAP). RFC 7252, IETF. <https://www.rfc-editor.org/rfc/rfc7252>
- [8] Connectivity Standards Alliance: Matter Specification. <https://csa-iot.org/all-solutions/matter/>